PYTHON 2

Iannick Gagnon
Baccalauréat en Génie mécanique (ÉTS)
Maîtrise et doctorat en Génie logiciel (ÉTS)
Professeur enseignant en informatique – **ÉTS, 2021**
Lead en machine learning – **Eli Health, 2023**

CONTACT

LinkedIn: https://tinyurl.com/5n6b2xtc

Couriel: iannick.gagnon@etsmtl.ca

Plan

- Notions intermédiaires
- Écrire du "vrai" code
- S'amuser

Tests: Assertions

```
assert expression_booleenne, message_si_faux
```

- Lance une erreur (AssertionError) lorsqu'une condition est fausse
  - Modèle mental: **Lorsqu'un test échoue**
- Pour **ce qui doit absolument être vrai**
- Peut être désactivé à l'exécution (**python -O**) ... sauf pour la NASA

https://fr.wikipedia.org/wiki/The_Power_of_10:_Rules_for_Developing_Safety-Critical_Code

```
>>> valeur_1 = int(input('Saisir un entier: '))
Saisir un entier: 1
>>> valeur_2 = int(input('Saisir un entier différent: '))
Saisir un entier différent: 1
>>> assert valeur_1 != valeur_2, 'Les valeurs doivent être différentes'
AssertionError: Les valeurs doivent être différentes
```

XR6 – Assertion d'égalité

Implémentez une fonction nommée **assert_equal** qui reçoit:

1. Une valeur obtenue (x_{obtenu}).
2. Une valeur cible (x_{cible}).
3. Une tolérance (ε) avec une valeur par défaut de 10^{-6}.

Celle-ci lance **une erreur d'assertion** si l'expression suivante est fausse :

$$x_{cible} - \varepsilon \leq x_{obtenu} \leq x_{cible} + \varepsilon$$

Tests: Librairie Pytest

- Riche en fonctionnalités: https://docs.pytest.org/en/stable/reference/reference.html
- Plus **flexible** et **complet** que les assertions seules

```
import pytest

liste_1 = [1, 2, 3, 4, 5]

# Créer une liste presque identique
liste_2 = [1.000001, 2, 3, 4, 5]

# Ceci ne déclenche pas d'erreur
assert liste_1 == pytest.approx(liste_2), 'Erreur (epsilon = 1e-6)'

# Changer la tolérange pour déclencher une erreur
assert liste_1 == pytest.approx(liste_2, abs=1e-7), 'Erreur (epsilon = 1e-7)'
```

```
AssertionError: Erreur (epsilon = 1e-7)
```

Exceptions

- Mécanisme qui permet de **signaler qu'un problème est survenu** pendant l'exécution (**raise**)
- Interrompt l'exécution (et remonte jusqu'au **try-except** le plus proche)

```
1 try:
2   raise ValueError("J'ai lancé la ValueError!")
3 except ValueError as err:
4   raise ValueError("J'ai attrapé la ValueError!") from err
5 print('Ce message ne sera pas affiché')
```

- L'instruction '**as** err' (ligne 3) stocke l'erreur (un objet) dans la variable 'err'
- L'instruction '**from e**' (ligne 4) permet de conserver l'origine

VOIR L'EXEMPLE DANS LE CARNET

PRENEZ LE TEMPS DE LIRE LE MESSAGE D'ERREUR!

XR6 – Gestion d'exceptions

- Dans la fonction **calculer_afficher_racines()**, levez une exception de type **ValueError** si le coefficient a est **nul**.
- Dans la fonction **main()**, faites appel à **calculer_afficher_racines()** avec $a = 0$ dans un bloc **try-except**.
  - Signalez qu'une erreur est survenue en levant une **RuntimeError**.
  - Préservez l'origine de l'erreur.

Une toute petite classe

- Implémenter ces propres types avec **class**
- Hériter des attributs et de méthodes d'une classe de base avec l'**héritage**

```
class TresGrosseErreur(Exception):
  """
  Erreur à lever lorsque Iannick écrit du code.
  """
  pass
```

- Le nom s'écrit en **CamelCase**: **class NomClasse(NomClasseBase)**
- Ici, **pass** signifie que nous n'avons rien à ajouter à la classe de base

```
>>> raise TresGrosseErreur('Oops!')
TresGrosseErreur: Oops!
```

Une classe-contenant

```python
from dataclasses import dataclass

@dataclass
class Vecteur:
  """
  Un vecteur en 1, 2 ou 3 dimensions.

  Attributs:
    dx (float): Le composant en x du vecteur. Valeur par défaut de 0.0.
    dy (float): Le composant en y du vecteur. Valeur par défaut de 0.0.
    dz (float): Le composant en z du vecteur. Valeur par défaut de 0.0.
  """
  dx: float = 0.0
  dy: float = 0.0
  dz: float = 0.0
```

```
>>> Vecteur(1.0, 2.0)
Vecteur(dx=1.0, dy=2.0, dz=0.0)
```

Valeur par défaut

Ajouter une méthode publique

```python
from dataclasses import dataclass

@dataclass
class Vecteur:

  [...]

  def calculer_norme(self) -> float:
    """
    Calcule la norme Euclidienne du vecteur.

    Args:
      Aucuns.

    Retourne:
      float: La norme du vecteur.
    """
    return (self.dx ** 2 + self.dy ** 2 + self.dz ** 2) ** 0.5
```

Référence implicite

```
>>> un_vecteur = Vecteur(1.0, 2.0)
>>> un_vecteur.calculer_norme()
2.23606797749979
```

XR6 – Vecteur direction

1. Créez une classe-exception nommée **NullNormError**.
2. Ajoutez une méthode nommée **calculer_vecteur_direction()** à la classe Vecteur qui retourne un nouveau vecteur unitaire ($\hat{v}$) dans la direction de ce dernier.
   1. Levez une erreur de type **NullNormError** si la norme du vecteur est nulle.
   2. Utilisez la fonction **approx** de **pytest** pour le test d'égalité de manière que l'erreur soit levée lorsque $\|\vec{v}\| < 10^{-12}$, qui est la tolérance par défaut de **approx()**.

Le vecteur direction se calcule comme suit:

$$\hat{v} = \frac{\vec{v}}{\|\vec{v}\|} = \left\langle \frac{v_x}{\|\vec{v}\|}, \frac{v_y}{\|\vec{v}\|}, \frac{v_z}{\|\vec{v}\|} \right\rangle$$

Sérialisation et persistence

- Transformer une donnée en un format qui peut être **stocké**, **lu**, ou transmis.

```
>>> vecteur = Vecteur(1.0, 2.0, 3.0)
>>> vecteur.sauvegarder_txt('vecteur.txt')
```

écriture

```
1.0,2.0,3.0
```

vecteur.txt

lecture

```
>>> vecteur_charge = Vecteur.charger_txt('vecteur.txt')
>>> print(vecteur_charge)
Vecteur(dx=1.0, dy=2.0, dz=3.0)
```

Ouvrir un fichier

- Un fichier s'ouvre avec la fonction native **open**:

[redacted]

```
with open(nom_fichier [, mode='r']) as fichier:
  ...
```

Mode	Description
'r'	Lecture (défaut)
'w'	Écriture
'r+' et 'w+'	Lecture et écriture
'a'	Ajouter à la fin
'b'	Binaire

Écrire dans un fichier

- Nous utilisons la méthode **write()** sur l'objet-fichier :

```
1 with open('fichier.txt', mode='w') as fichier:
2   for i in range(5):
3     fichier.write(f'Ligne no.{i + 1}: bla-bla\n')
```

```
1 Ligne no.1: bla-bla
2 Ligne no.2: bla-bla
3 Ligne no.3: bla-bla
4 Ligne no.4: bla-bla
5 Ligne no.5: bla-bla
```

fichier.txt

Lire un fichier

- Nous utilisons la méthode **readlines()** sur l'objet-fichier :

```
1  with open('fichier.txt', mode='r') as fichier:
2    texte: list[str] = fichier.readlines()
3
4  for ligne in texte:
5    print(ligne)
```

```
1  Ligne no.1: bla-bla
2
3  Ligne no.2: bla-bla
4
5  Ligne no.3: bla-bla
6
7  Ligne no.4: bla-bla
8
9  Ligne no.5: bla-bla
10
```

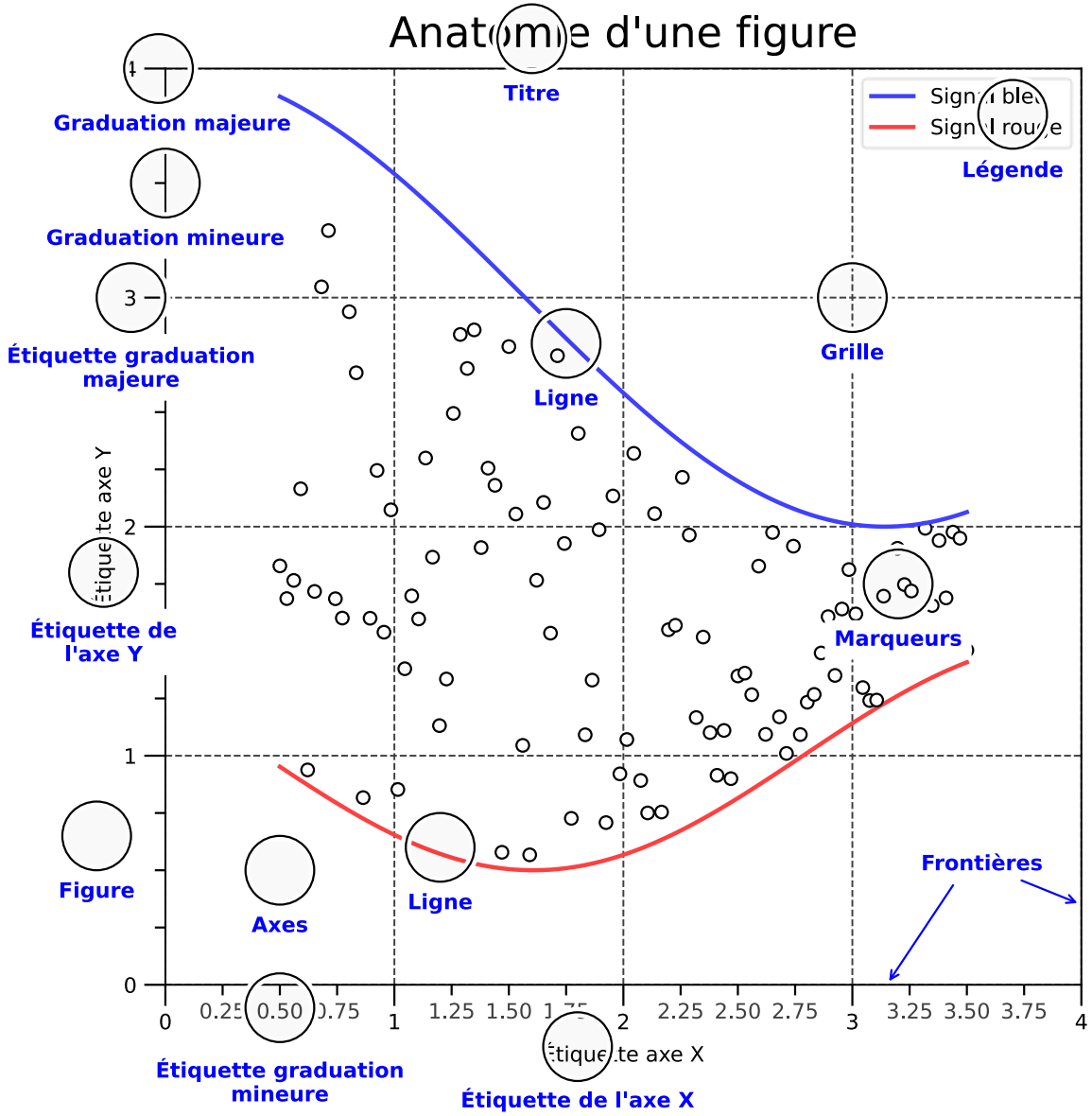
XR6 – Vecteur sérialisé

1. Ajoutez une méthode nommée **sauvegarder_txt()** qui reçoit le nom du fichier et qui stocke les composants du vecteur sur des lignes séparées.
2. Ajoutez une méthode nommée **charger_txt()** qui reçoit le nom du fichier et qui charge les composants du vecteur à partir de celui-ci.
    1. Assurez-vous qu'il y a bel et bien 3 composants de lus dans le fichier.
    2. N'oubliez par de convertir les composants lus en rationnels avec la fonction **float()**.

VOIR LE CODE-EXÉCUTEUR DANS LE CARNET

Bilan

- Nous sommes capables de ...
    - **Définir des types/classes** et d'en **créer des instances** sous la forme d'**objets.**
    - Structurer rapidement les **propriétés/attributs** (**@dataclass**).
    - Ajouter des comportements (**méthodes**).
    - **Lire** et **écrire** des **fichiers** (I/O).
    - **Sérialiser** et **désérialiser** des données.
- Prochain arrêt: **Visualiser des données.**



Un graphique est un objet composé d'objets ...

Graphique point-ligne

- Importer l'interface **pyplot** de **matploblib** (inspirée de MATLAB)

```
import matploblib.pyplot as plt
```

- Tracer un graphique

```
[figure =] plt.plot(x, y, [format])
```

- Afficher le graphique en mode non-interactif

```
plt.show([figure])
```

Notation en compréhension

- En maths (angl. ***set-builder notation***):

$$\{x^2 | x \in \mathbb{N}, x \text{ est pair}, x \leq 10\}$$

$$\downarrow$$

$$\{0, 2, 4, 6, 8, 10\}$$

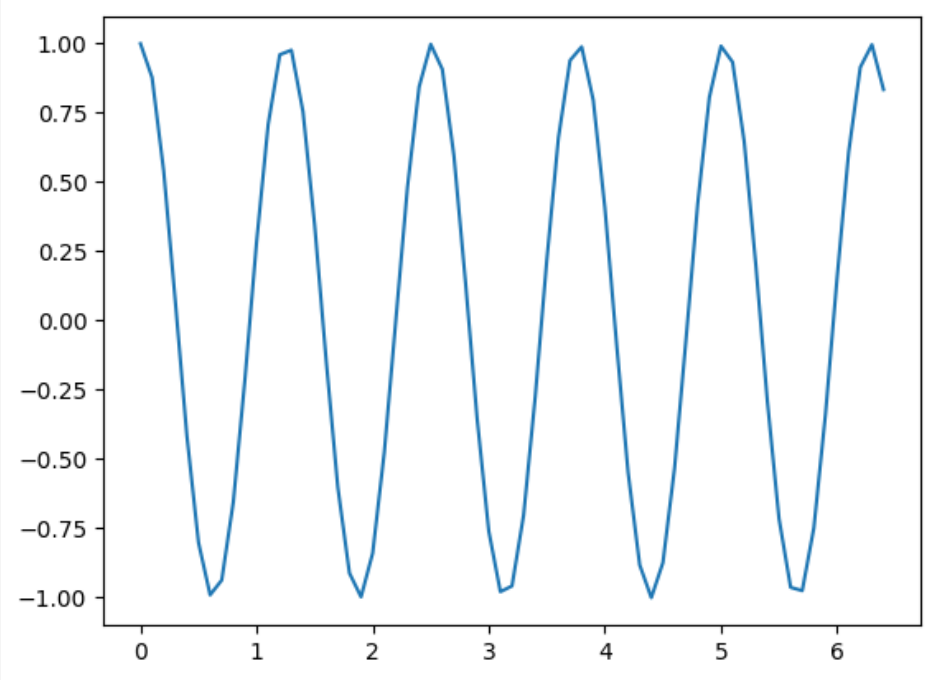
- En Python:

```
>>> [x for x in range(11) if x % 2 == 0 and <= 10]
[0, 2, 4, 6, 8, 10]
```

- S'applique aux liste, tuples ... et même aux dictionnaires!

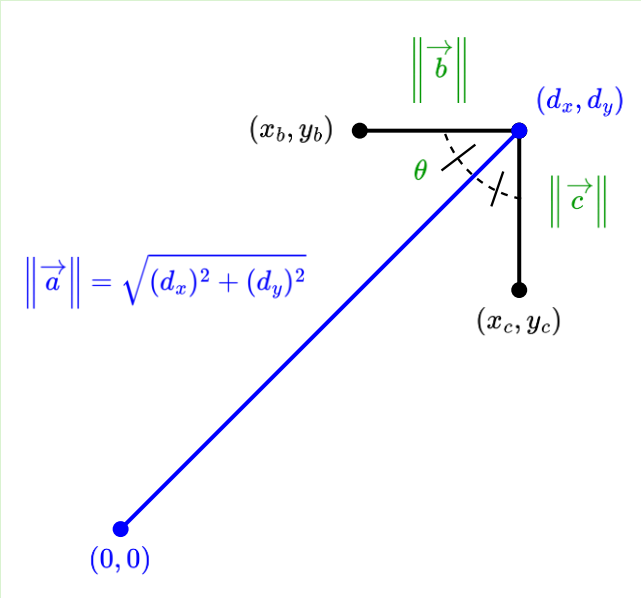
Exemple

```python
# Importer l'interface pyplot de matplotlib
import matplotlib.pyplot as plt

# Fonction cosinus
from math import cos

# Créer les données à tracer
x: list[float] = [0.1 * i for i in range(65)]
y: list[float] = [cos(5 * valeur) for valeur in x]

# Créer le graphique
plt.plot(x, y)

# L'afficher
plt.show()
```

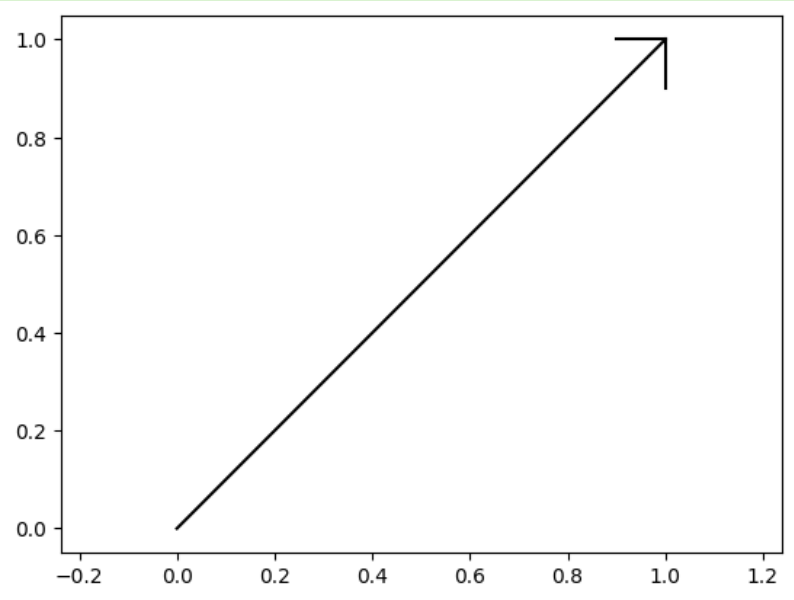


XR6 – Tracer des vecteurs

Complétez la méthode **`tracer_xy()`** dans le carnet du cours. Cette dernière récupère les coordonnées (x_b, y_b) et (x_c, y_c) avec la méthode **`_calculer_coordonnees_fleche()`** puis s'en sert pour afficher un graphique qui schématise le vecteur avec une tête en flèche.



paramétrisation



graphique

Pour aller plus loin

- Pendant l'atelier, vous découvrirez comment ajouter un titre, nommer les axes, etc.
- Consultez la page d'exemples: https://matplotlib.org/stable/gallery/index.html
- Lisez gratuitement l'excellent livre intitulé **Scientific Visualization: Python + Matplotlib** de Nicolas Rougier disponible ici: https://github.com/rougier/scientific-visualization-book

Temps d'exécution (1/3)

```python
from time import perf_counter

debut: float = perf_counter()

# Le code à mesurer

fin: float = perf_counter()

# Calculer et afficher la durée
duree: float = fin - debut
print(f'Temps d'exécution: {duree} s')
```

VOIR L'EXEMPLE DANS LE CARNET

Temps d'exécution (2/3)

- La méthode précédente varie d'un essai à l'autre.
- La méthode **`repeat()`** de la librairie **timeit** permet de faire plusieurs essais pour calculer des statistiques. Cependant, elle nécessite souvent une **expression lambda**.

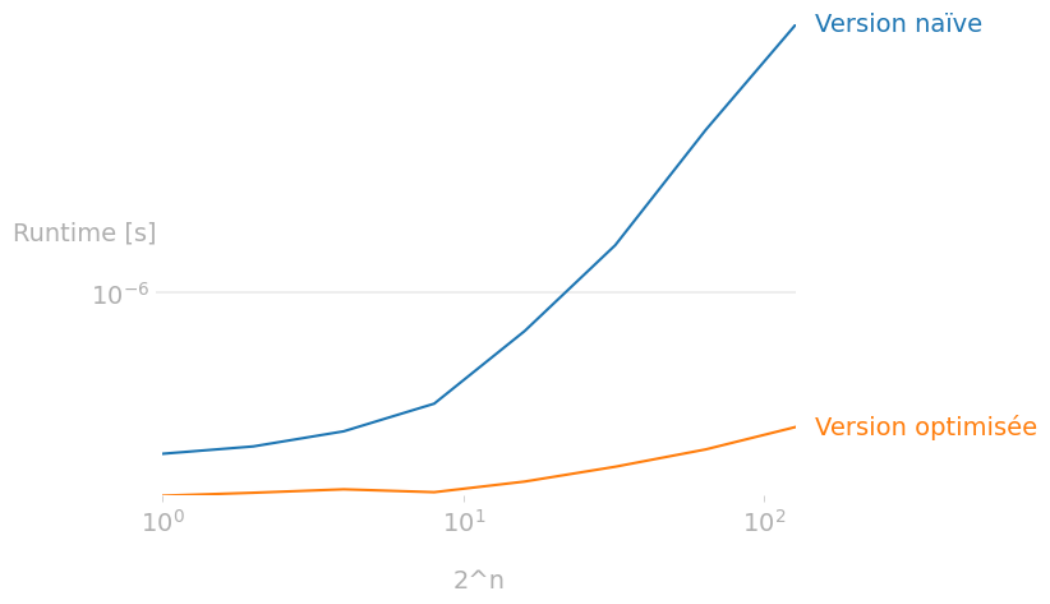
```
lambda [paramètres], expression
```

```
>>> foo_puissance = lambda base, puissance: base ** puissance
>>> foo_puissance(2, 7)
128
```

VOIR L'EXEMPLE DANS LE CARNET

Temps d'exécution (3/3)

- Visualiser la performance avec **perfplot**:



VOIR L'EXEMPLE DANS LE CARNET

Si le temps le permet ...

CSV, JSON et YAML

- Formats optimisés selon les usages:
  - **CSV**: Données tabulaires (ex: Excel) → **`import csv`**
  - **JSON**: Données structurées (ex: dictionnaires) → **`import json`**
  - **YAML**: Souvent utilisé pour des configurations → **`import yaml`**

`fichier.csv`

```
1,2,3,4,5
```

`fichier.json`

```
{
  "vecteur": {
    "dx": 3,
    "dy": 4,
    "norme": 5
  }
}
```

`fichier.yaml`

```
options:
  number: 1
  repeat: 3
```

VOIR LES EXEMPLES DANS LE CARNET